

Customer Feedback Analysis

End-to-End Sentiment Analysis Project — Build Guide

About This Project

This guide walks through building a complete customer-feedback sentiment analysis system from scratch in Python. Starting from Amazon product reviews, you set up a clean project structure, load the data and derive a sentiment label from each review's own text (not the star rating), clean the review text using NLP techniques (lemmatization and stop-word removal with spaCy), convert text into numerical features with TF-IDF, then train and evaluate a Logistic Regression model that classifies each review as positive, neutral, or negative.

By the end you will have a trained model, a command-line prediction script, and a FastAPI web service exposing a `/predict` endpoint that a frontend team can call, plus an automated Telegram alert that messages the business owner the moment a review is classified negative. The project is organized into a series of phases, each building on the last.

What you'll build: a project scaffold, a data-loading and preprocessing pipeline, a TF-IDF feature pipeline (with an optional Word2Vec embedding pipeline), a trained sentiment classifier, an evaluation report, an interactive prediction tool, a deployable REST API, and an automated Telegram alert for negative reviews.

Business Understanding

LIFECYCLE STAGE 1 · BUSINESS UNDERSTANDING

Every project starts here: before a line of code, get clear on the problem and on how success will actually be delivered. The business owner is buried in product reviews and cannot read them all. The ones that matter most are the complaints — a bad batch, a slow delivery, a quality issue — because those are what the owner can act on. Lost in a sea of happy reviews, they are easy to miss.

The goal: read every incoming review, decide whether it is positive, neutral, or negative, and the moment one is negative, push it straight to the business owner on Telegram. No dashboard to check, no report to open — the complaint lands on the owner's phone in seconds, while it still matters.

How the finished system works, end to end:

- A new review arrives — typed into the prediction tool, or sent to the `/predict` API.
- The text is cleaned exactly the way the model was trained to expect (Phase 6).
- The model classifies it as positive, neutral, or negative (Phases 8–9).
- If the verdict is negative, a Telegram message with the review is sent to the owner automatically (Phase 13).

- The owner reads it and acts — with no one manually scanning reviews.

This is why the whole build is tuned around catching negatives rather than chasing overall accuracy. A model that is 95% accurate but quietly misses half the complaints fails the business; one that reliably surfaces complaints succeeds. That single priority drives later choices — the balanced class weighting in Phase 9 and the focus on negative-class recall in Phase 10.

From here, each phase below carries a small tag showing which machine-learning lifecycle stage it belongs to — Data Collection, Data Understanding, Data Preparation, Model Building, Training, Testing, and Evaluation. Stages 1 to 8 (through Model Evaluation) are the data scientist's and ML engineer's work and live in Phases 1 to 11. Model Deployment (Phase 12) is the clean hand-off to the full-stack or DevOps team, and CI/CD — automating the release pipeline — stays with them. Phase 13 then wires up the Telegram alerts that deliver the business goal defined right here.

Topics Coverage Map

Every concept from the NLP Foundations, Preprocessing & Vectorization syllabus and the phase of this guide where you put it into practice. The last two rows are foundational concepts behind vectorization — this guide implements the TF-IDF approach rather than their raw-count versions.

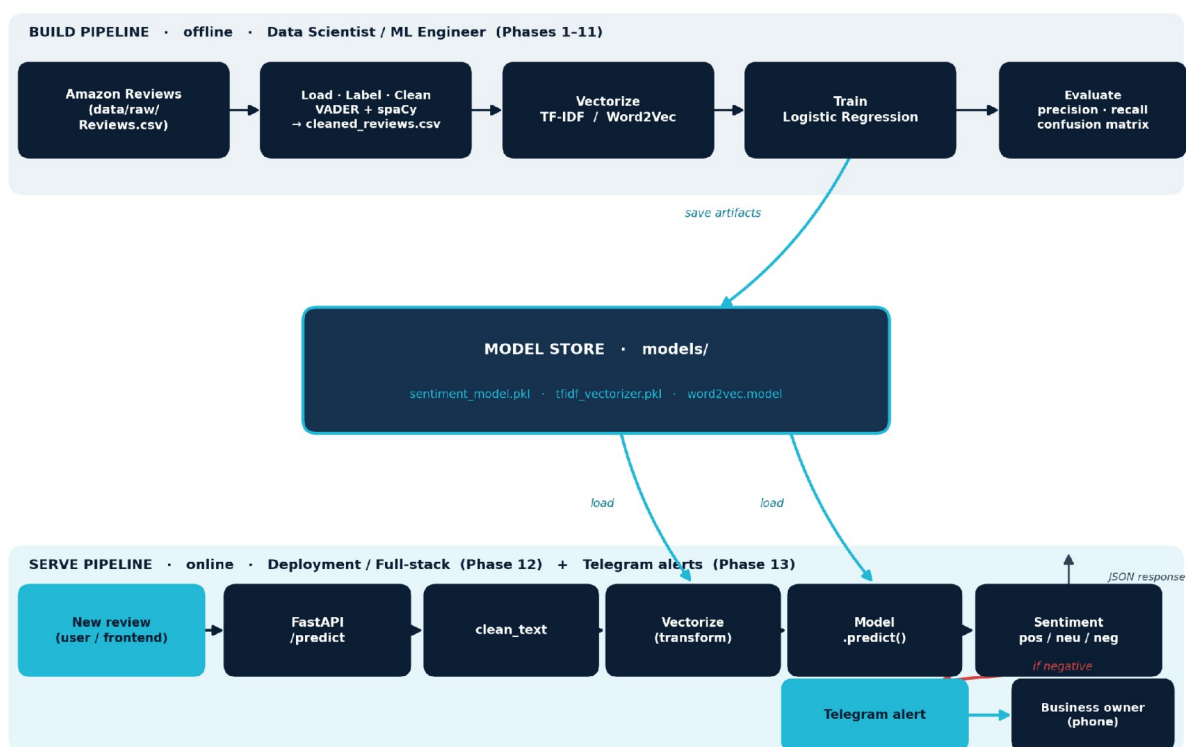
Topic	Where	How it's covered
Introduction to NLP	About, Ph 6	Overview of turning raw, messy review text into machine-ready numerical features.
NLP Pipeline	Phase 7	<code>process_dataset()</code> runs the full label-then-clean pipeline over the dataset in one pass.
Text Cleaning & Lower Casing	Phase 6	<code>str(text).lower()</code> plus regex cleaning inside <code>clean_text()</code> normalize the text.
Punctuation & Stop Words	Phase 6	<code>re.sub(r"[^a-zA-Z]", ...)</code> strips punctuation; <code>token.is_stop</code> removes common stop-words.
Tokenization	Phase 6	<code>nlp(text)</code> runs spaCy, which splits each review into individual tokens.
Stemming & Lemmatization	Phase 6	<code>token.lemma_</code> reduces each word to its base form (lemmatization, the more precise of the two).
Preprocessing Pipeline	Ph 5–7	Sentiment labeling (Ph 5) and text cleaning (Ph 6) combined and applied to the whole dataset (Ph 7).
Corpus & Vocabulary	Phase 8	<code>TfidfVectorizer</code> learns the vocabulary from the cleaned corpus, capped at <code>max_features=10000</code> .
N-Grams	Phase 8	<code>ngram_range=(1,2)</code> captures both single words and two-word phrases as features.
TF · IDF · TF-IDF	Phase 8	<code>TfidfVectorizer</code> converts text into TF-IDF-weighted numeric vectors for the model.

Topic	Where	How it's covered
Bag of Words	Concept (Ph 8)	The foundation behind vectorization; this guide uses the weighted TF-IDF form instead of plain counts.
Count Vectorizer	Concept (Ph 8)	The raw-count cousin of TF-IDF; this guide applies TF-IDF, which down-weights very common words.
Word Embeddings (Word2Vec)	Phase 8B	Dense vectors learned from how words co-occur; an alternative to TF-IDF that captures word meaning and similarity (e.g. “great” near “excellent”).

System Architecture

SYSTEM DESIGN · HOW THE PIECES FIT

Before building anything, here is the shape of the whole system. It is two pipelines that meet at one shared model store. The build pipeline runs offline and turns raw reviews into a trained model saved on disk. The serve pipeline runs online: it loads that saved model to classify live reviews, and fires a Telegram alert to the owner whenever one comes back negative.



End-to-end architecture — the offline build pipeline (top) trains and saves the model; the online serve pipeline (bottom) loads it to classify incoming reviews and alert the owner on negatives.

Two pipelines, one model store

The build pipeline is the data scientist's work (Phases 1–11). Reviews are loaded, labelled with VADER, and cleaned with spaCy; the cleaned text is vectorised with TF-IDF (or Word2Vec); a Logistic Regression model is trained and evaluated. The trained model and the fitted vectorizer are then saved into the `models/` folder — the single source of truth the rest of the system depends on.

The serve pipeline is the online path (Phase 12 onward). A new review arrives at the FastAPI `/predict` endpoint, is cleaned by the exact same `clean_text` used in training, transformed by the saved vectorizer, and scored by the saved model. The sentiment is returned as JSON — and if it is negative, `send_negative_alert` pushes the review to the business owner on Telegram (Phase 13).

Note: Both pipelines must clean and vectorise text the same way — a model only makes sense on features built exactly as it was trained. The model store is what guarantees that: the serve pipeline never re-derives anything, it loads the very vectorizer and model the build pipeline saved.

How the code is organised

Each box above maps to a folder in the project, so the architecture and the repository line up one to one:

- **src/data/** — loading, VADER labelling, and spaCy cleaning (Phases 4–7).
- **src/features/** — the TF-IDF and Word2Vec vectorizers (Phases 8, 8B).
- **src/models/** — training, evaluation, and the interactive predictor (Phases 9–11).
- **src/api/** — the FastAPI service that exposes `/predict` (Phase 12).
- **src/alerts/** — the Telegram alert module (Phase 13).
- **models/** — the saved artifacts both pipelines share.
- **data/** — the raw source data and the processed, cleaned dataset.

The data flow, one line each:

```
Training:  Reviews.csv -> clean -> vectorize -> train -> models/
Serving:   review -> /predict -> clean -> vectorize -> model -> sentiment
           -> (if negative) -> Telegram -> business owner
```

Phase 1: Project Setup

SETUP · ENVIRONMENT

Create the project folder.

Why: A dedicated folder keeps every file for this project in one isolated, organized place.

```
mkdir customer_feedback_analysis
cd customer_feedback_analysis
```

Create virtual environment.

Mac/Linux

Why: A virtual environment isolates this project's libraries so they don't clash with other Python projects on your machine.

```
python3 -m venv .venv
```

Activate:

Why: Activating makes every install and run command use this project's isolated Python instead of the system one.

```
source .venv/bin/activate
```

Windows

Why: Same isolation step for Windows — create the environment, then activate it.

```
python -m venv .venv  
.
```

Install Libraries

Create:

Why: Listing dependencies in one file lets anyone recreate the exact same environment with a single command.

```
requirements.txt
```

Paste:

Why: These are the libraries the project relies on — data handling (pandas, numpy), machine learning (scikit-learn), NLP (nltk, spacy), the web API (fastapi, uvicorn), and plotting (matplotlib, seaborn).

```
pandas  
numpy  
scikit-learn  
nltk  
spacy  
regex  
joblib  
fastapi  
uvicorn  
matplotlib  
seaborn  
gensim  
requests
```

Install:

Why: This installs everything listed in requirements.txt in one go.

```
pip install -r requirements.txt
```

Install SpaCy model:

Why: spaCy needs its language model downloaded separately; this English model powers the tokenizing and lemmatization used later.

```
python -m spacy download en_core_web_sm
```

Download VADER lexicon:

Why: We label sentiment from the review text using NLTK's VADER analyzer; this one-time download fetches the lexicon it scores words against.

```
python -m nltk.downloader vader_lexicon
```

Phase 2: Create Folder Structure

SETUP · PROJECT STRUCTURE

Inside project:

Why: A clear layout separates data, models, notebooks, and source code so the project stays maintainable as it grows.

```
customer_feedback_analysis/
```

```
├── data/
│   ├── raw/
│   ├── processed/
│   └── sample/
├── models/
├── notebooks/
├── src/
│   ├── api/
│   ├── data/
│   ├── features/
│   ├── models/
│   └── utils/
├── requirements.txt
├── README.md
└── main.py
```

Create them manually or:

Why: These commands build the whole structure automatically instead of creating each folder by hand.

```
mkdir -p data/raw
mkdir -p data/processed
mkdir -p data/sample

mkdir -p src/api
mkdir -p src/data
```

```
mkdir -p src/features
mkdir -p src/models
mkdir -p src/utils
```

```
mkdir models
mkdir notebooks
```

Phase 3: Add Kaggle Dataset

LIFECYCLE STAGE 2 · DATA COLLECTION

Download:

Why: The model learns from real review data, so we need a genuine labeled dataset to work with.

Amazon Product Reviews Dataset

Put it here:

Why: Keeping the untouched original in a dedicated raw folder means we always have the source data to fall back to.

```
data/raw/Reviews.csv
```

Example:

```
customer_feedback_analysis/
data/
├── raw/
│   └── Reviews.csv
```

Phase 4: Data Loading

LIFECYCLE STAGE 3 · DATA UNDERSTANDING

Create file:

Why: We keep the data-loading logic in its own module so it can be reused and tested independently.

```
src/data/load_data.py
```

Paste:

Why: This small function reads the CSV into a pandas DataFrame — the starting point for everything that follows.

```
# Lifecycle stage 3 – Data Understanding
import pandas as pd

def load_data(path):
    return pd.read_csv(path)
```

Test It

Create:

Why: main.py is the entry point we use to run and verify each piece as we build it.

```
main.py
```

Why: Loading the data and printing the first rows confirms the file path and column names are correct before moving on.

```
from src.data.load_data import load_data

df = load_data("data/raw/Reviews.csv")

print(df.head())
print(df.columns)
```

Run:

```
python main.py
```

Expected:

```
Score
Summary
Text
```

You should see first 5 rows.

Phase 5: Create Sentiment Labels

LIFECYCLE STAGE 4 · DATA PREPARATION

Create:

Why: A model needs a target to learn from, so we derive a sentiment label from each review's text using VADER (a rule-based analyzer) instead of relying on the star rating.

```
src/data/preprocess.py
```

Why: VADER reads the raw text and returns a compound polarity score from -1 to +1; we map ≥ 0.05 to positive, ≤ -0.05 to negative, and everything in between to neutral.

```
# Lifecycle stage 4 – Data Preparation (sentiment labels)
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer

try:
    nltk.data.find("sentiment/vader_lexicon.zip")
except LookupError:
    nltk.download("vader_lexicon")

analyzer = SentimentIntensityAnalyzer()
```



```
def create_sentiment(text):

    score = analyzer.polarity_scores(str(text))["compound"]

    if score >= 0.05:
        return "positive"

    elif score <= -0.05:
        return "negative"

    else:
        return "neutral"
```

Test It

Update main.py

***Why:** We apply the labeling to the dataset and spot-check that each review's text maps to a sensible sentiment.*

```
from src.data.load_data import load_data
from src.data.preprocess import create_sentiment

df = load_data("data/raw/Reviews.csv")

df["sentiment"] = df["Text"].apply(
    create_sentiment
)

print(
    df[["Text", "sentiment"]].head()
)
```

Run:

```
python main.py
```

Expected:

	Text	sentiment
0	I have bought several of the Vita...	positive
1	Product arrived labeled as Jumbo ...	negative
2	This is a confection that has bee...	positive
3	If you are looking for the secret...	neutral

Phase 6: NLP Text Cleaning

LIFECYCLE STAGE 4 · DATA PREPARATION

Open:

***Why:** Raw review text is messy — mixed case, punctuation, URLs, and filler words — so we clean it to help the model focus on meaningful words.*

Add:

Why: This function lowercases the text, strips URLs and non-letters, removes stop-words, and reduces each word to its base form (lemma) — but deliberately keeps negation words like “not” and “never”, since dropping them would turn “not good” into “good” and flip the sentiment.

```
# Lifecycle stage 4 – Data Preparation (text cleaning)
import re
import spacy

nlp = spacy.load("en_core_web_sm")

# Negation words carry sentiment, so we never drop them
KEEP = {"not", "no", "never", "nor", "none", "nt"}

def clean_text(text):
    text = str(text).lower()

    text = re.sub(r"http\S+", "", text)

    text = re.sub(r"^[a-zA-Z ]", "", text)

    doc = nlp(text)

    words = []

    for token in doc:
        if token.lemma_ in KEEP or not token.is_stop:
            words.append(token.lemma_)

    return " ".join(words)

def clean_batch(texts):
    cleaned = []

    # nlp.pipe streams the texts through spaCy in batches;
    # disabling parser + ner skips the slow components we don't use
    docs = nlp.pipe(
        texts,
        disable=["parser", "ner"],
        batch_size=200
    )

    for doc in docs:
        words = [
            t.lemma_ for t in doc
            if t.lemma_ in KEEP or not t.is_stop
        ]
```

```
cleaned.append(" ".join(words))

return cleaned
```

Why two functions: *clean_text* cleans one review at a time (used by the prediction script and the API). *clean_batch* does the same work but streams thousands of reviews through spaCy in batches and skips the parser and NER components — this is what makes processing tens of thousands of rows take minutes instead of hours. We use *clean_batch* for the full dataset in Phase 7.

Note: If you ever change *clean_text* after already building the project, you must re-run Phase 7 (process), Phase 8 (vectorize), and Phase 9 (train). The saved vocabulary and model are baked from the old cleaning, so predictions stay wrong until you rebuild them.

Test Cleaning

Update *main.py*

Why: *Running it on a sample sentence confirms the cleaning produces tidy, normalized output before we apply it to the whole dataset.*

```
from src.data.preprocess import clean_text

sample = """
This product is AMAZING!!!
I bought 2 units.
"""

print(clean_text(sample))
```

Run:

```
python main.py
```

Expected:

```
product amazing buy unit
```

Phase 7: Process Entire Dataset

LIFECYCLE STAGE 4 · DATA PREPARATION

Create:

Why: *This ties the previous steps together — labeling and cleaning the whole dataset in one pass and saving the result so we only do this slow work once.*

```
src/data/process_dataset.py
```

Why: *Wrapping it in a function with print checkpoints makes the pipeline reusable and easy to follow as it runs. We take a random 50,000-row sample and clean it with *clean_batch*, which keeps the run to roughly 3–6 minutes; on a first run, drop 50000 to 1000 to confirm everything works before committing to the full sample.*

```
# Lifecycle stage 4 – Data Preparation (full dataset)
import pandas as pd

from src.data.preprocess import create_sentiment, clean_batch

def process_dataset():

    print("Script Started")

    df = pd.read_csv("data/raw/Reviews.csv")

    print("Dataset Loaded")

    # Random sample, NOT head() – the file is grouped by product,
    # so the first rows are a narrow slice. random_state keeps it repeatable.
    df = df.sample(50000, random_state=42)

    df["sentiment"] = df["Text"].apply(create_sentiment)

    print("Sentiment Created")

    # Lowercasing + regex are fast as vectorized pandas ops,
    # so we do them here and leave only tokenizing/lemmatizing to spaCy.
    texts = df["Text"].astype(str).str.lower()
    texts = texts.str.replace(r"http\S+", "", regex=True)
    texts = texts.str.replace(r"[^a-zA-Z ]", "", regex=True)

    df["clean_review"] = clean_batch(texts)

    print("Cleaning Completed")

    df.to_csv(

        "data/processed/cleaned_reviews.csv",

        index=False

    )

    print("File Saved")
```

Test Processed Dataset

Update main.py

Why: Calling the function from main.py runs the full labeling-and-cleaning pipeline end to end.

```
from src.data.process_dataset import process_dataset

process_dataset()
```

Run:

```
python main.py
```

Output:

Dataset processed

Check:

***Why:** Saving the cleaned data means later steps can load it instantly instead of re-running the cleaning every time.*

```
data/processed/  
cleaned_reviews.csv
```

created successfully.

Phase 8: TF-IDF Vectorization

LIFECYCLE STAGE 5 · MODEL BUILDING

Create:

***Why:** Machine-learning models need numbers, not text, so we turn each review into a TF-IDF vector that reflects how important each word is.*

```
src/features/vectorizer.py
```

***Why:** This fits the vectorizer on the cleaned reviews and saves it, so the exact same transformation can be reused at prediction time.*

```
# Lifecycle stage 5 – Model Building (TF-IDF features)  
import pandas as pd  
import joblib  
  
from sklearn.feature_extraction.text import (TfidfVectorizer)  
  
df = pd.read_csv("data/processed/cleaned_reviews.csv")  
  
vectorizer = TfidfVectorizer(max_features=10000, ngram_range=(1,2))  
  
X = vectorizer.fit_transform( df["clean_review"] )  
  
joblib.dump( vectorizer, "models/tfidf_vectorizer.pkl" )  
  
print(X.shape)
```

Run

```
python src/features/vectorizer.py
```

Output:

```
(50000, 10000)
```

Example.

Check:

```
models/  
tfidf_vectorizer.pkl
```

Phase 8B: Word2Vec Vectorization (Alternative to TF-IDF)

LIFECYCLE STAGE 5 · MODEL BUILDING

TF-IDF in Phase 8 treats every word as its own isolated column and never learns that “great” and “excellent” mean almost the same thing. Word2Vec takes a different route: it learns a dense vector for each word from the company that word keeps, so words used in similar contexts end up close together. This phase adds Word2Vec as an alternative feature pipeline. TF-IDF stays the default used by Phases 9 to 12; Word2Vec is saved to its own files so the two never overwrite each other and you can compare them.

Install gensim:

Why: gensim is the standard Python library for training Word2Vec embeddings. It is already in the updated requirements.txt from Phase 1 — install it directly if you skipped that step.

```
pip install gensim
```

Create:

Why: We keep the Word2Vec logic in its own feature module, mirroring the TF-IDF vectorizer so either can be swapped in.

```
src/features/word2vec_vectorizer.py
```

Paste:

Why: This adds document_vector — which turns a whole review into one vector by averaging its word vectors — and a train_word2vec() function that learns the embeddings from the cleaned reviews. The training sits under if __name__ == “__main__” so that importing document_vector elsewhere does not kick off a full retrain.

```
# Lifecycle stage 5 – Model Building (Word2Vec features)  
import pandas as pd  
import numpy as np  
  
from gensim.models import Word2Vec  
  
def document_vector(model, text):  
    words = str(text).split()  
  
    # keep only words the model actually learned  
    vectors = [  
        model.wv[w]  
        for w in words  
        if w in model.wv  
    ]
```

```

# empty or all-unknown review -> zero vector
if not vectors:
    return np.zeros(model.vector_size)

return np.mean(vectors, axis=0)

def train_word2vec():

    df = pd.read_csv(
        "data/processed/cleaned_reviews.csv"
    )

    df["clean_review"] = df["clean_review"].fillna("")

    # Word2Vec learns from lists of tokens,
    # so we split each cleaned review into words.
    sentences = [
        review.split()
        for review in df["clean_review"]
    ]

    # vector_size = length of each word vector
    # window      = neighbouring words used as context
    # min_count    = ignore words seen fewer than twice
    # workers      = CPU cores used to train
    w2v = Word2Vec(
        sentences,
        vector_size=200,
        window=5,
        min_count=2,
        workers=4,
        epochs=10
    )

    w2v.save("models/word2vec.model")

    print(
        "Word2Vec Trained:",
        len(w2v.wv.index_to_key),
        "words"
    )

if __name__ == "__main__":
    train_word2vec()

```

Run:

```
python src/features/word2vec_vectorizer.py
```

Output:

```
Word2Vec Trained: 18452 words
```

Example — your exact word count will differ.

Train on Word2Vec Features

Create:

Why: This mirrors Phase 9 but feeds the classifier dense Word2Vec document-vectors instead of sparse TF-IDF columns, and saves to a separate file so your TF-IDF model stays intact.

```
src/models/train_w2v.py
```

Paste:

Why: We rebuild every review as one averaged 200-dimension vector, stack them into a feature matrix, then train the same balanced Logistic Regression and save it alongside the Word2Vec model.

```
# Lifecycle stage 6 – Model Training (Word2Vec)
import pandas as pd
import numpy as np
import joblib

from gensim.models import Word2Vec

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from src.features.word2vec_vectorizer import document_vector

df = pd.read_csv( "data/processed/cleaned_reviews.csv")

df["clean_review"] = df["clean_review"].fillna("")

w2v = Word2Vec.load("models/word2vec.model")

# Turn every review into one averaged 200-d vector
X = np.vstack([
    document_vector(w2v, text)
    for text in df["clean_review"]
])

y = df["sentiment"]

X_train, X_test, y_train, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )
)

model = LogisticRegression(
    class_weight="balanced",
    max_iter=1000
)

model.fit(X_train, y_train)
```



```
joblib.dump(  
    model,  
    "models/sentiment_model_w2v.pkl"  
)  
  
print("Word2Vec Model Trained")
```

Run:

```
python src/models/train_w2v.py
```

Output:

```
Word2Vec Model Trained
```

TF-IDF vs Word2Vec — which to use

Both turn text into numbers, but they trade off differently:

- **TF-IDF** — sparse, fast, and surprisingly strong for sentiment on a dataset this size. It keys on the exact words that signal sentiment (“refund”, “broke”, “love”), which is often all you need. This is why it stays the default for Phases 9 to 12.
- **Word2Vec** — dense and meaning-aware: it can tell that “awful” and “terrible” are related even if one barely appears in training. It tends to shine on larger datasets and on text where wording varies a lot, though the simple averaging used here can blur strong single-word signals.

To use the Word2Vec model in the prediction script or the FastAPI service, load `models/word2vec.model` and `models/sentiment_model_w2v.pkl`, run the review through `clean_text`, then through `document_vector` instead of `vectorizer.transform`. Keep one path live at a time so the model always sees features built the same way it was trained.

Note: Word2Vec features are dense floats, so never mix them with the saved TF-IDF vectorizer — a model trained on one will give meaningless predictions on the other. Always pair `word2vec.model` with `sentiment_model_w2v.pkl`, and `tfidf_vectorizer.pkl` with `sentiment_model.pkl`.

Phase 9: Train Model

LIFECYCLE STAGE 6 · MODEL TRAINING

Create:

Why: This trains the actual classifier that learns to predict sentiment from the vectorized reviews.

```
src/models/train.py
```

Why: We split the data into train and test sets, fit a Logistic Regression model, then save both the model and the vectorizer so they can be loaded later without retraining. We pass `class_weight="balanced"` because the reviews skew heavily positive; without it the model defaults to predicting positive for almost everything.

```

# Lifecycle stage 6 – Model Training
import pandas as pd
import joblib

from sklearn.model_selection import (
    train_test_split
)

from sklearn.feature_extraction.text import (
    TfidfVectorizer
)

from sklearn.linear_model import (
    LogisticRegression
)

df = pd.read_csv(
    "data/processed/cleaned_reviews.csv"
)

X = df["clean_review"]
y = df["sentiment"]

vectorizer = TfidfVectorizer(
    max_features=10000,
    ngram_range=(1,2)
)

X = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )
)

model = LogisticRegression(
    class_weight="balanced",
    max_iter=1000
)

model.fit(
    X_train,
    y_train
)

joblib.dump(
    model,
    "models/sentiment_model.pkl"
)

joblib.dump(
    vectorizer,

```

```
    "models/tfidf_vectorizer.pkl"
)
print("Training Completed")
```

Run Training

```
python src/models/train.py
```

Output:

```
Training Completed
```

Phase 10: Evaluate Model

LIFECYCLE STAGE 8 · MODEL EVALUATION

Create:

***Why:** Before trusting the model, we measure how accurately it predicts on data it never saw during training.*

```
src/models/evaluate.py
```

***Why:** Loading the saved model and vectorizer and scoring the held-out test set tells us how well it really performs.*

```
# Lifecycle stage 8 – Model Evaluation
import pandas as pd
import joblib

from sklearn.metrics import (
    classification_report,
    confusion_matrix
)

from sklearn.model_selection import (
    train_test_split
)

df = pd.read_csv(
    "data/processed/cleaned_reviews.csv"
)

model = joblib.load(
    "models/sentiment_model.pkl"
)

vectorizer = joblib.load(
    "models/tfidf_vectorizer.pkl"
)

X = vectorizer.transform(
```

```

    df["clean_review"]
)
y = df["sentiment"]
_, X_test, _, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )
)

predictions = model.predict(X_test)

print(
    classification_report(
        y_test,
        predictions
    )
)

# Fix the label order so the grid is readable and the
# negative class always sits in the top-left corner.
labels = ["negative", "neutral", "positive"]

cm = confusion_matrix(
    y_test,
    predictions,
    labels=labels
)

cm_table = pd.DataFrame(
    cm,
    index=["actual_negative", "actual_neutral", "actual_positive"],
    columns=["pred_negative", "pred_neutral", "pred_positive"]
)

print("\nConfusion Matrix (rows = actual, columns = predicted)")
print(cm_table)

```

Why: The classification report gives summary scores, but the confusion matrix shows the raw counts behind them — exactly which reviews were classified correctly and which were confused for another class. We fix the label order to negative, neutral, positive so the negative class always lands in the top-left corner, where it is easy to read off the numbers that matter most for this project.

Run:

```
python src/models/evaluate.py
```

Expected:

Why: Precision, recall, and F1-score show how well the model does for each sentiment class, not just overall.

```
precision
recall
f1-score
```

Reading the Confusion Matrix for Our Business Problem

For this project, surfacing every positive review does not help the owner much — happy customers do not tell the owner what to fix. The value is in the **negative** reviews: complaints about a product, quality, or delivery are exactly what the owner needs to catch and act on. So the question we really care about is: of all the truly negative reviews, how many did the model actually catch?

The confusion matrix answers that directly. The example grid below shows the layout:

```
Confusion Matrix (rows = actual, columns = predicted)
               pred_negative  pred_neutral  pred_positive
actual_negative      1820         340         210
actual_neutral       180         1450         290
actual_positive      120         260         4860
```

The **actual_negative row** is the one to watch, because it is about real complaints:

- **1820 (top-left)** — complaints the model correctly caught and flagged as negative. This is the win: real negative feedback reaches the owner. (For the negative class, these are the True Positives.)
- **340 + 210 = 550 (rest of that row)** — negative reviews the model missed, mislabeling them as neutral or positive. These are the costly misses: unhappy customers slip through silently and the owner never learns about the problem. (These are the False Negatives, and they are the number we most want to drive down.)
- **180 + 120 (rest of the pred_negative column)** — neutral or positive reviews wrongly flagged as negative. These are just noise: the owner reads a few extra reviews that turn out fine. (These are False Positives — far less harmful here than a missed complaint.)

In short, success for this project is a high count in the top-left cell and a low total across the rest of the negative row — in other words, **high recall on the negative class**. A model with 95% overall accuracy that quietly buries half the complaints is useless for the owner. This is also why we set `class_weight="balanced"` in Phase 9: the data skews heavily positive, and without it the model plays it safe by calling almost everything positive, missing the very negatives the owner depends on.

Phase 11: Prediction Script

LIFECYCLE STAGE 7 · MODEL TESTING

Create:

Why: This gives a quick, interactive way to try the trained model on your own typed reviews.

```
src/models/predict.py
```

Why: It loads the saved model and vectorizer, then loops so you can keep entering reviews and instantly see the predicted sentiment. Each review is passed through `clean_text` first, so the input is processed the exact same way as the training data — without this, the model sees a different kind of text than it learned from and predictions become unreliable.

```
# Lifecycle stage 7 – Model Testing
import joblib
from src.data.preprocess import clean_text

model = joblib.load(
    "models/sentiment_model.pkl"
)

vectorizer = joblib.load(
    "models/tfidf_vectorizer.pkl"
)

while True:
    review = input(
        "\nEnter Review: "
    )

    X = vectorizer.transform(
        [clean_text(review)]
    )

    prediction = model.predict(X)

    print(
        "\nPrediction:",
        prediction[0]
    )
```

Run:

```
python src/models/predict.py
```

Test:

```
Enter Review:
This product is amazing

Prediction:
positive
```

Phase 12: FastAPI (For Fullstack Team)

LIFECYCLE STAGE 9 · MODEL DEPLOYMENT (HAND-OFF)

Create:

Why: Wrapping the model in a FastAPI service lets other applications — like a frontend — request predictions over HTTP.

```
src/api/app.py
```

***Why:** This defines a `/predict` endpoint that takes review text, cleans it with `clean_text` (the same step used in training), runs it through the saved vectorizer and model, and returns the sentiment as JSON.*

```
# Lifecycle stage 9 – Model Deployment (hand-off)
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
from src.data.preprocess import clean_text

app = FastAPI()

model = joblib.load(
    "models/sentiment_model.pkl"
)

vectorizer = joblib.load(
    "models/tfidf_vectorizer.pkl"
)

class Review(BaseModel):
    text: str

@app.post("/predict")
def predict(review: Review):

    X = vectorizer.transform(
        [clean_text(review.text)]
    )

    sentiment = model.predict(X)[0]

    return {
        "sentiment": sentiment
    }
```

Run:

***Why:** Starting the server makes the `/predict` endpoint live on your machine.*

```
uvicorn src.api.app:app --reload
```

Open:

***Why:** FastAPI auto-generates interactive docs, so you can test the endpoint right in the browser without writing any client code.*

```
http://127.0.0.1:8000/docs
```

Phase 13: Automated Negative-Review Alerts (Telegram)

BUSINESS OUTCOME · NEGATIVE-REVIEW ALERTS

This is the payoff from Business Understanding. The model can now label a review, but the owner only gets value when a complaint reaches them. Here we close that loop: the moment a review is classified negative, the system messages the owner on Telegram automatically — so complaints arrive in seconds instead of being buried.

Create a Telegram bot

Why: A Telegram bot is the cheapest, fastest way to push a message to a phone. @BotFather hands you a bot token; your chat id tells the bot where to send.

In Telegram, message @BotFather, send /newbot, follow the prompts, and copy the bot token it gives you.

Then send any message to your new bot, open the URL below in a browser, and read your chat id from the JSON (the value at result[0].message.chat.id):

```
https://api.telegram.org/bot<YOUR_TOKEN>/getUpdates
```

Store the credentials safely

Why: Secrets never belong in source code. We read the token and chat id from environment variables so the code is safe to share and commit.

Mac/Linux:

```
export TELEGRAM_BOT_TOKEN="your-bot-token"
export TELEGRAM_CHAT_ID="your-chat-id"
```

Windows (PowerShell):

```
setx TELEGRAM_BOT_TOKEN "your-bot-token"
setx TELEGRAM_CHAT_ID "your-chat-id"
```

Install the requests library (it is in the updated requirements.txt from Phase 1):

```
pip install requests
```

Create the alert module

Why: One small module with one job: given a review, message the owner. Keeping it separate means both the prediction script and the API can reuse it.

```
src/alerts/telegram_alert.py
```

First make the folder:

```
mkdir src/alerts
```

Paste:

Why: send_negative_alert builds a short message, posts it to Telegram's sendMessage endpoint with your chat id, and raises an error if the call fails so problems are not silently swallowed.


```

# Lifecycle stage – Business outcome (negative-review alerts)
import os
import requests

# Read secrets from the environment, never hard-code them
TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
CHAT_ID = os.environ["TELEGRAM_CHAT_ID"]

API_URL = f"https://api.telegram.org/bot{TOKEN}/sendMessage"

def send_negative_alert(review):

    message = (
        "NEGATIVE REVIEW ALERT\n\n"
        f"{review}"
    )

    payload = {
        "chat_id": CHAT_ID,
        "text": message
    }

    response = requests.post(
        API_URL,
        data=payload,
        timeout=10
    )

    response.raise_for_status()

```

Fire the alert on every negative prediction

***Why:** This is what makes it automatic. We extend the Phase 12 API so that every review hitting /predict is checked — if the model calls it negative, the owner is messaged before the response is even returned.*

Update src/api/app.py:

```

# Lifecycle stage 9 – Model Deployment, now firing the business outcome
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
from src.data.preprocess import clean_text
from src.alerts.telegram_alert import send_negative_alert

app = FastAPI()

model = joblib.load(
    "models/sentiment_model.pkl"
)

vectorizer = joblib.load(
    "models/tfidf_vectorizer.pkl"
)

```

```

class Review(BaseModel):
    text: str

@app.post("/predict")
def predict(review: Review):

    X = vectorizer.transform(
        [clean_text(review.text)]
    )

    sentiment = model.predict(X)[0]

    # Business outcome: push negatives to the owner instantly
    if sentiment == "negative":
        send_negative_alert(review.text)

    return {"sentiment": sentiment}

```

Run the server and post a negative review to /predict — the owner's Telegram should light up within a second or two:

```
uvicorn src.api.app:app --reload
```

Note: This fires on real, incoming reviews through the API — exactly the flow described in Business Understanding. If you also want to sweep the existing dataset and alert on historical complaints, loop over the rows where sentiment is negative and call `send_negative_alert` on each, but watch Telegram's rate limits (roughly 30 messages per second) and consider batching so you do not flood the owner.